

Fields and Watches

DCGM represents inventory attributes, gauges, counters, events, and profiling metrics as *fields*. A client creates a *watch* when it asks the host engine to collect selected fields for selected entities on a schedule. The resulting timestamped samples are shared host-engine state.

This model is the foundation for direct monitoring and for higher-level features such as health, policy, profiling, and process or job statistics. Begin with [Entities and Groups](#) if entity families and groups are not yet familiar.

Fields Define the Data

A field definition has a numeric ID and metadata that describes the value. The metadata includes:

Entity level

The family for which the value is meaningful, such as GPU, MIG instance, NVSwitch, NVLink, CPU, or CPU core.

Scope

Whether the value belongs to an entity or is global to the host engine.

Value type

Integer, floating-point, string, timestamp, or binary.

Semantics

Whether the value is an attribute, instantaneous gauge, cumulative counter, event, or another kind of measurement.

A *field sample* adds runtime context to that definition:

```
(entity family, entity ID, field ID, timestamp, value, status)
```

The timestamp is when DCGM collected the value, not when a client later read it. The status distinguishes a usable value from conditions such as not supported, not found, no permission, or not yet available.

Find fields

List the fields known to the installed command-line client:

```
$ dcgmi dmon --list
```

The output gives each field’s numeric ID, short column name, and description. It is a metadata catalogue, not a hardware probe. A listed field can still be unavailable for a particular entity family, GPU generation, driver, or platform.

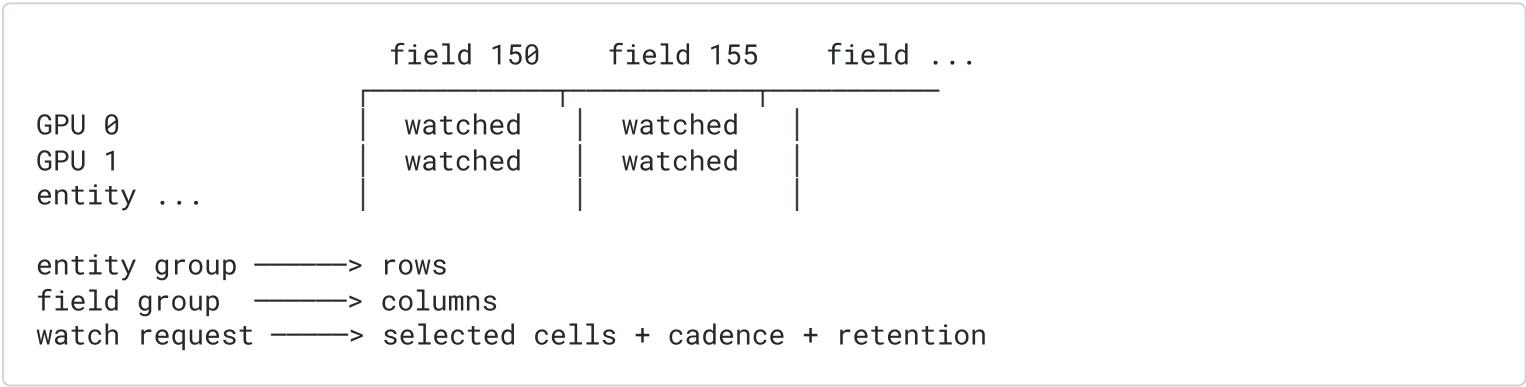
Use [Field Identifiers](#) for the complete definitions and [Field Scope](#) before pairing a field with a non-GPU entity. Do not infer units or counter behavior from a short `dmon` column name.

Watches Produce Samples

A watch combines four choices:

- the entities to observe;
- the fields to collect;
- the requested update interval; and
- retention limits expressed as age and/or sample count.

Entity groups and field groups are reusable selectors on the two axes of a watch:



A field group does not start collection, and an entity group does not imply that fields are watched. The watch is the operation that combines them.

First sample and sample age

The first value might not exist until the next collection cycle. A client that needs to wait for an update must use the API’s update/wait behavior or allow a complete interval before reading. Always evaluate the returned timestamp:

- a recent sample can still describe a counter accumulated over a much longer device lifetime;
- an old sample can remain readable until its retention limit expires; and
- reading the latest sample normally does not force a new provider query.

Shared collection

The host engine tracks watchers independently but collects one stream for a given entity-field pair. If several clients request that same pair, DCGM uses the fastest requested interval and the longest requested retention needed by the active watchers. Removing one client's watch does not stop another client's request.

For example, if one client requests GPU 0 power every five seconds and another requests it every second, the effective collection interval is one second. Both clients read from the retained stream and share the underlying provider collection through the host engine.

Sharing does not make telemetry free. Resource use can grow when an administrator:

- watches more entity-field combinations;
- requests shorter intervals;
- retains samples for longer;
- subscribes more clients to updates; or
- transfers larger result sets.

Provider and hardware limits can also make a requested interval impractical or enforce a minimum. Choose a cadence based on how quickly an operator can act on the value, not merely on the fastest accepted number.

Retention is an operational buffer

Retain enough data for the consumer to read it and for the intended analysis window to complete. A scheduler job summary, for example, requires samples from before the job began through its end. A client that requests only the latest value needs much less history.

The cache is held in host memory. It is bounded by watch limits, can discard old samples, and is lost when the host engine exits. Export samples to a time-series or accounting system when they need durable retention.

Field Groups Reuse a Selection

A field group is a named list of field IDs stored in the running host engine. It is useful when several operations or clients use the same telemetry set. It does not select entities, set a cadence, choose retention, or validate support on a particular device.

Create a field group for GPU temperature (150) and power usage (155):

```
$ dcgmi fieldgroup --create gpu-environment --fieldids 150,155
```

The command prints the numeric field-group ID. Field-group names and IDs are unique within one host engine. A user-created field group remains after the `dcgmi` process exits and until it is deleted

or the host engine exits. Internal field groups used by DCGM can be listed but not deleted.

Inspect and remove the selection with:

```
$ dcgmi fieldgroup --info --fieldgroup <field-group-id>
$ dcgmi fieldgroup --delete --fieldgroup <field-group-id>
```

Deleting a field group prevents future use of that selector. It does not necessarily stop compatible watches that another client or module already owns.

Monitor from the Command Line

`dcgmi dmon` is an interactive consumer of the field service. It selects fields directly or through a field group, selects entities directly or through an entity group, and displays samples at the requested delay.

Use direct selections for a short investigation:

```
$ dcgmi dmon --field-id 150,155 \
  --entity-id gpu:0,gpu:1 --count 10
```

Use reusable selections for a repeated workflow:

```
$ dcgmi dmon --field-group-id <field-group-id> \
  --group-id <group-id> --delay 5000 --count 12
```

The delay is in milliseconds. `dmon` requests connection-scoped watches and at most two retained samples for its own use. When it exits, its watch requests end. The entity and field groups remain because they are host-engine objects, and other clients' compatible watches remain active.

For a long-running integration, use the field APIs to choose update frequency, maximum age, maximum sample count, and either latest-value or history retrieval. `dmon` is not a durable collector.

Interpret Values Correctly

`N/A` is a presentation, not one diagnosis

`dcgmi dmon` renders binary values and all DCGM blank sentinel values as `N/A`. Depending on the field and row, this can mean:

- the field is not supported for that hardware or entity family;
- its provider or required module is unavailable;

- the host engine lacks permission;
- the field has not produced its first sample;
- the entity is no longer accessible; or
- the value cannot be represented in `dmon` output.

Check the entity, field scope, module, permissions, and sample timing. Use the programmatic field-value status when an integration must preserve the exact reason.

Gauges and counters answer different questions

A gauge such as temperature or power describes a sampled state. Compare its timestamp and cadence with the event being investigated.

A cumulative counter describes activity since the provider's reset boundary, not necessarily since the watch began. Record a baseline before an interval and subtract it from the value afterward. A counter that is unchanged during the interval is historical context; a counter that increased needs interpretation against workload, topology, and health information.

An event field can be sparse. Retention must cover the period between the event and the consumer's read.

Design a Monitoring Set

For a new monitoring integration:

1. Define the operator question, such as thermal headroom or link errors.
2. Discover the target entities and confirm their durable identity.
3. Look up fields, entity scope, units, and counter semantics.
4. Choose the slowest cadence that still meets the response requirement.
5. Retain enough samples for collection delays and the analysis window.
6. Test unsupported hardware, no-permission, and first-sample behavior.
7. Measure host-engine overhead with the intended entity and field counts.
8. Recreate watches and groups after a host-engine restart.

Further Reading

- [dcgmi dmon](#) defines field selection, display cadence, output, and command-line watch lifetime.
- [dcgmi fieldgroup](#) defines field-group limits and management.
- [Field APIs](#) documents programmatic watches and cached-value retrieval.

... and [this](#) for more information on the DPM and DPM API.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.